



Hello, &
WELCOME
To ALL.

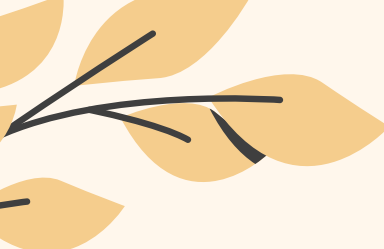


MySQL BASE

HOSPITAL MANAGEMENT SYSTEM

Presenting By Amit Shinde





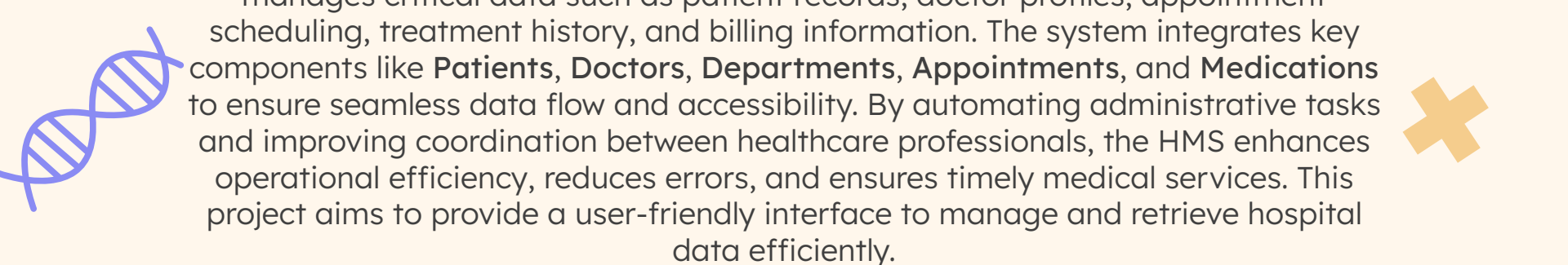
AIM OF PROJECT

- Data Management Efficiency:
 - Appointment Scheduling and Tracking:
 - Patient Treatment and Medication Management:
 - Doctor-Patient Interaction and Care Coordination:
 - Hospital Department and Resource Allocation:
- 
- 



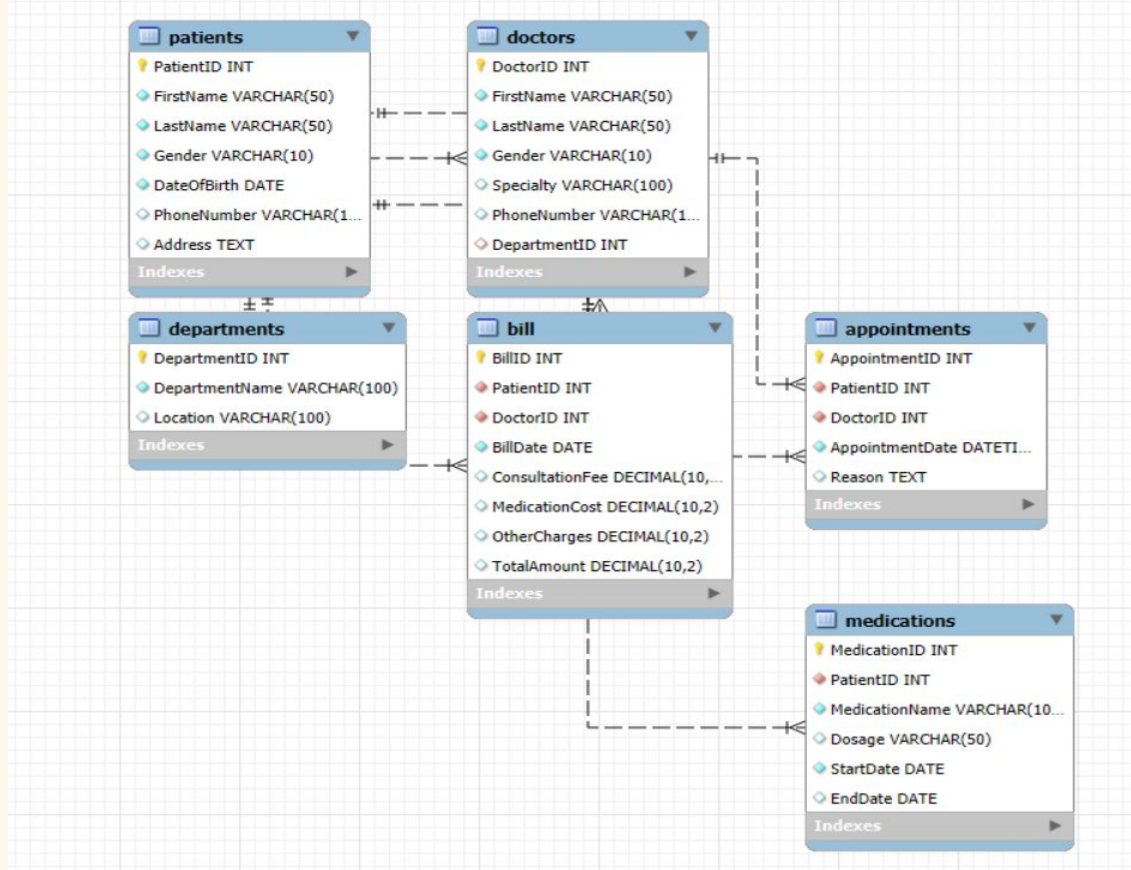


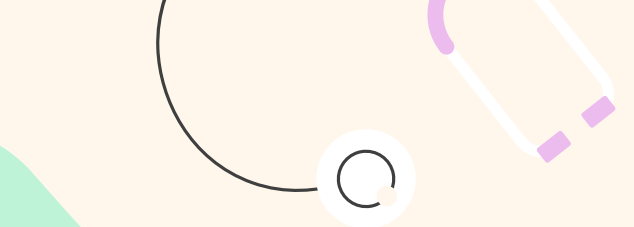
Introduction



The **Hospital Management System (HMS)** is a comprehensive database solution designed to streamline hospital operations and improve patient care. It efficiently manages critical data such as patient records, doctor profiles, appointment scheduling, treatment history, and billing information. The system integrates key components like **Patients, Doctors, Departments, Appointments, and Medications** to ensure seamless data flow and accessibility. By automating administrative tasks and improving coordination between healthcare professionals, the HMS enhances operational efficiency, reduces errors, and ensures timely medical services. This project aims to provide a user-friendly interface to manage and retrieve hospital data efficiently.

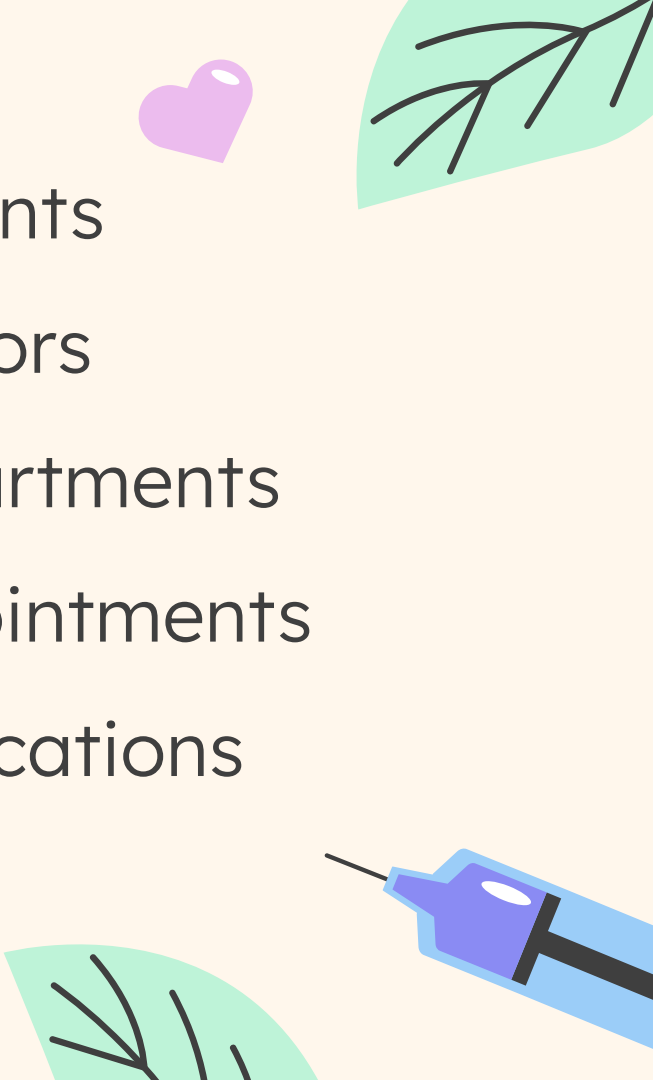
ENTITY RELATIONSHIP DIAGRAM





Tables In Project



- Patients
 - Doctors
 - Departments
 - Appointments
 - Medications
 - Bill
- 

Description Of Tables



Table Patients

-- Table: Patients

```
CREATE TABLE Patients (  
    PatientID INT AUTO_INCREMENT PRIMARY KEY,  
    FirstName VARCHAR(50) NOT NULL,  
    LastName VARCHAR(50) NOT NULL,  
    Gender VARCHAR(10) NOT NULL, -- 'Male', 'Female', 'Other'  
    DateOfBirth DATE NOT NULL,  
    PhoneNumber VARCHAR(15),  
    Address TEXT  
);  
DESC Patients;
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 

	Field	Type	Null	Key	Default	Extra
▶	PatientID	int	NO	PRI	NULL	auto_increment
	FirstName	varchar(50)	NO		NULL	
	LastName	varchar(50)	NO		NULL	
	Gender	varchar(10)	NO		NULL	
	DateOfBirth	date	NO		NULL	
	PhoneNumber	varchar(15)	YES		NULL	
	Address	text	YES		NULL	

Table Doctors

```
-- Table: Doctors
CREATE TABLE Doctors (
  DoctorID INT AUTO_INCREMENT PRIMARY KEY,
  FirstName VARCHAR(50) NOT NULL,
  LastName VARCHAR(50) NOT NULL,
  Gender VARCHAR(10) NOT NULL, -- 'Male', 'Female', 'Other'
  Specialty VARCHAR(100),
  PhoneNumber VARCHAR(15),
  DepartmentID INT,
  FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
);
DESC Doctors;
```

```
DESC Doctors;
```

```
);
```

```
FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)
```

Field	Type	Null	Key	Default	Extra
DoctorID	int	NO	PRI	NULL	auto_increment
FirstName	varchar(50)	NO		NULL	
LastName	varchar(50)	NO		NULL	
Gender	varchar(10)	NO		NULL	
Specialty	varchar(100)	YES		NULL	
PhoneNumber	varchar(15)	YES		NULL	
DepartmentID	int	YES	MUL	NULL	

```
DepartmentID
```

```
int
```

```
YES
```

```
MUL
```

```
NULL
```

PhoneNumber	varchar(15)	YES		NULL	
Specialty	varchar(100)	YES		NULL	
Gender	varchar(10)	NO		NULL	
LastName	varchar(50)	NO		NULL	
FirstName	varchar(50)	NO		NULL	
DoctorID	int	NO	PRI	NULL	auto_increment

Table Departments

```
-- Table: Departments  
  
CREATE TABLE Departments (  
    DepartmentID INT AUTO_INCREMENT PRIMARY KEY,  
    DepartmentName VARCHAR(100) NOT NULL,  
    Location VARCHAR(100)  
);  
  
DESC Departments;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Field	Type	Null	Key	Default	Extra
▶	DepartmentID	int	NO	PRI	NULL	auto_increment
	DepartmentName	varchar(100)	NO		NULL	
	Location	varchar(100)	YES		NULL	

Table Appointments

-- Table: Appointments

```
CREATE TABLE Appointments (  
  AppointmentID INT AUTO_INCREMENT PRIMARY KEY,  
  PatientID INT NOT NULL,  
  DoctorID INT NOT NULL,  
  AppointmentDate DATETIME NOT NULL,  
  Reason TEXT,  
  FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),  
  FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)  
);
```

Result Grid		Filter Rows:	Export: 	Wrap Cell Content: 		
	Field	Type	Null	Key	Default	Extra
▶	AppointmentID	int	NO	PRI	NULL	auto_increment
	PatientID	int	NO	MUL	NULL	
	DoctorID	int	NO	MUL	NULL	
	AppointmentDate	datetime	NO		NULL	
	Reason	text	YES		NULL	

Table Medications

-- Table: Medications

```
CREATE TABLE Medications (  
  MedicationID INT AUTO_INCREMENT PRIMARY KEY,  
  PatientID INT NOT NULL,  
  MedicationName VARCHAR(100) NOT NULL,  
  Dosage VARCHAR(50),  
  StartDate DATE NOT NULL,  
  EndDate DATE,  
  FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)  
);
```

```
DESC Medications;
```

```
DESC Medications;
```

```
)?
```

```
FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)
```

Result Grid		Filter Rows:	Export: 	Wrap Cell Content: 		
	Field	Type	Null	Key	Default	Extra
▶	MedicationID	int	NO	PRI	NULL	auto_increment
	PatientID	int	NO	MUL	NULL	
	MedicationName	varchar(100)	NO		NULL	
	Dosage	varchar(50)	YES		NULL	
	StartDate	date	NO		NULL	
	EndDate	date	YES		NULL	

The background is a light beige color. It is decorated with stylized leaves in light green and yellowish-orange, and blue DNA double helix structures. There are also small plus signs: a purple one in the upper left and a blue one in the lower right.

INSERTING

In Departments Table

-- Insert records in department table

- **INSERT INTO** Departments (DepartmentName, Location) **VALUES**
('Cardiology', 'Building A'),
('Neurology', 'Building B'),
('Orthopedics', 'Building C'),
('Pediatrics', 'Building D'),
('Radiology', 'Building E'),
('Oncology', 'Building F'),
('Dermatology', 'Building G'),
('Gastroenterology', 'Building H'),
('Psychiatry', 'Building I'),
('Urology', 'Building J');
- **SELECT * FROM** Departments;

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

DepartmentID	DepartmentName	Location
1	Cardiology	Building A
2	Neurology	Building B
3	Orthopedics	Building C
4	Pediatrics	Building D
5	Radiology	Building E
6	Oncology	Building F
7	Dermatology	Building G
8	Gastroenterology	Building H
9	Psychiatry	Building I
10	Urology	Building J
* NULL	* NULL	* NULL

In Doctors Table

-- Insert records in Doctors table

- **INSERT INTO** Doctors (FirstName, LastName, Gender, Specialty, PhoneNumber, DepartmentID) **VALUES**
('John', 'Doe', 'Male', 'Cardiologist', '1234567890', 1),
('Jane', 'Smith', 'Female', 'Neurologist', '2345678901', 2),
('Robert', 'Brown', 'Male', 'Orthopedic Surgeon', '3456789012', 3),
('Emily', 'Davis', 'Female', 'Pediatrician', '4567890123', 4),
('Michael', 'Miller', 'Male', 'Radiologist', '5678901234', 5),
('Sarah', 'Wilson', 'Female', 'Oncologist', '6789012345', 6),
('James', 'Taylor', 'Male', 'Dermatologist', '7890123456', 7),
('Laura', 'Anderson', 'Female', 'Gastroenterologist', '8901234567', 8),
('David', 'Thomas', 'Male', 'Psychiatrist', '9012345678', 9),
('Sophia', 'Harris', 'Female', 'Urologist', '0123456789', 10);
- **SELECT * FROM** Doctors;

• **SELECT * FROM** Doctors;

```
( 'John', 'Doe', 'Male', 'Cardiologist', '1234567890', 1),  
( 'Jane', 'Smith', 'Female', 'Neurologist', '2345678901', 2),  
( 'Robert', 'Brown', 'Male', 'Orthopedic Surgeon', '3456789012', 3),  
( 'Emily', 'Davis', 'Female', 'Pediatrician', '4567890123', 4),  
( 'Michael', 'Miller', 'Male', 'Radiologist', '5678901234', 5),  
( 'Sarah', 'Wilson', 'Female', 'Oncologist', '6789012345', 6),  
( 'James', 'Taylor', 'Male', 'Dermatologist', '7890123456', 7),  
( 'Laura', 'Anderson', 'Female', 'Gastroenterologist', '8901234567', 8),  
( 'David', 'Thomas', 'Male', 'Psychiatrist', '9012345678', 9),  
( 'Sophia', 'Harris', 'Female', 'Urologist', '0123456789', 10);
```

DoctorID	FirstName	LastName	Gender	Specialty	PhoneNumber	DepartmentID
1	John	Doe	Male	Cardiologist	1234567890	1
2	Jane	Smith	Female	Neurologist	2345678901	2
3	Robert	Brown	Male	Orthopedic Surgeon	3456789012	3
4	Emily	Davis	Female	Pediatrician	4567890123	4
5	Michael	Miller	Male	Radiologist	5678901234	5
6	Sarah	Wilson	Female	Oncologist	6789012345	6
7	James	Taylor	Male	Dermatologist	7890123456	7
8	Laura	Anderson	Female	Gastroenterologist	8901234567	8
9	David	Thomas	Male	Psychiatrist	9012345678	9
10	Sophia	Harris	Female	Urologist	0123456789	10
NULL	NULL	NULL	NULL	NULL	NULL	NULL

In Patients Table

-- Insert records in Patients table

- **INSERT INTO** Patients (FirstName, LastName, Gender, DateOfBirth, PhoneNumber, Address) **VALUES**
('Alice', 'Johnson', 'Female', '1985-03-15', '1234567890', '123 Main St'),
('Bob', 'Williams', 'Male', '1990-07-20', '2345678901', '456 Oak St'),
('Catherine', 'Jones', 'Female', '2000-12-10', '3456789012', '789 Pine St'),
('Daniel', 'Brown', 'Male', '1975-05-25', '4567890123', '101 Maple St'),
('Eve', 'Garcia', 'Female', '1988-09-05', '5678901234', '202 Birch St'),
('Frank', 'Martinez', 'Male', '1965-02-18', '6789012345', '303 Cedar St'),
('Grace', 'Rodriguez', 'Female', '1995-11-30', '7890123456', '404 Elm St'),
('Henry', 'Lee', 'Male', '1978-01-22', '8901234567', '505 Spruce St'),
('Isla', 'Walker', 'Female', '1999-04-14', '9012345678', '606 Aspen St'),
('Jack', 'Hall', 'Male', '1982-06-09', '0123456789', '707 Willow St');
• **SELECT * FROM** Patients;

SELECT * FROM Patients;

```
( 'Jack', 'Hall', 'Male', '1982-06-09', '0123456789', '707 Willow St'),  
( 'Isla', 'Walker', 'Female', '1999-04-14', '9012345678', '606 Aspen St'),
```

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	PatientID	FirstName	LastName	Gender	DateOfBirth	PhoneNumber	Address
▶	1	Alice	Johnson	Female	1985-03-15	1234567890	123 Main St
	2	Bob	Williams	Male	1990-07-20	2345678901	456 Oak St
	3	Catherine	Jones	Female	2000-12-10	3456789012	789 Pine St
	4	Daniel	Brown	Male	1975-05-25	4567890123	101 Maple St
	5	Eve	Garcia	Female	1988-09-05	5678901234	202 Birch St
	6	Frank	Martinez	Male	1965-02-18	6789012345	303 Cedar St
	7	Grace	Rodriguez	Female	1995-11-30	7890123456	404 Elm St
	8	Henry	Lee	Male	1978-01-22	8901234567	505 Spruce St
	9	Isla	Walker	Female	1999-04-14	9012345678	606 Aspen St
	10	Jack	Hall	Male	1982-06-09	0123456789	707 Willow St
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

*	10	Jack	Hall	Male	1982-06-09	0123456789	707 Willow St		
	8	Henry	Lee	Male	1978-01-22	8901234567	505 Spruce St		
	9	Isla	Walker	Female	1999-04-14	9012345678	606 Aspen St		

In Appointment Table

-- Insert records in Appointments table

- **INSERT INTO** Appointments (PatientID, DoctorID, AppointmentDate, Reason) **VALUES**
(1, 1, '2024-11-21 09:00:00', 'Routine check-up'),
(2, 2, '2024-11-21 10:00:00', 'Headache evaluation'),
(3, 3, '2024-11-21 11:00:00', 'Knee pain'),
(4, 4, '2024-11-21 12:00:00', 'Child vaccination'),
(5, 5, '2024-11-21 13:00:00', 'MRI scan'),
(6, 6, '2024-11-21 14:00:00', 'Cancer consultation'),
(7, 7, '2024-11-21 15:00:00', 'Skin rash'),
(8, 8, '2024-11-21 16:00:00', 'Stomach ache'),
(9, 9, '2024-11-21 17:00:00', 'Depression counseling'),
(10, 10, '2024-11-21 18:00:00', 'Bladder issues');
- **SELECT * FROM** Appointments;

Result Grid

Filter Rows:

Edit:

</

In Medications Table

-- Insert records in Medications table

- **INSERT INTO** Medications (PatientID, MedicationName, Dosage, StartDate, EndDate) **VALUES**
(1, 'Aspirin', '100 mg daily', '2024-11-01', '2024-11-30'),
(2, 'Ibuprofen', '200 mg twice daily', '2024-11-05', '2024-11-20'),
(3, 'Paracetamol', '500 mg every 6 hours', '2024-11-10', '2024-11-15'),
(4, 'Amoxicillin', '250 mg three times daily', '2024-11-12', '2024-11-22'),
(5, 'Metformin', '500 mg daily', '2024-11-18', '2024-12-18'),
(6, 'Atorvastatin', '10 mg daily', '2024-11-01', '2024-12-01'),
(7, 'Prednisone', '20 mg daily', '2024-11-03', '2024-11-13'),
(8, 'Omeprazole', '40 mg daily', '2024-11-08', '2024-11-28'),
(9, 'Citalopram', '20 mg daily', '2024-11-15', '2024-12-15'),
(10, 'Tamsulosin', '0.4 mg daily', '2024-11-10', '2024-11-30');
• **SELECT * FROM** Medications;

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Contents:

	MedicationID	PatientID	MedicationName	Dosage	StartDate	EndDate
▶	1	1	Aspirin	100 mg daily	2024-11-01	2024-11-30
	2	2	Ibuprofen	200 mg twice daily	2024-11-05	2024-11-20
	3	3	Paracetamol	500 mg every 6 hours	2024-11-10	2024-11-15
	4	4	Amoxicillin	250 mg three times daily	2024-11-12	2024-11-22
	5	5	Metformin	500 mg daily	2024-11-18	2024-12-18
	6	6	Atorvastatin	10 mg daily	2024-11-01	2024-12-01
	7	7	Prednisone	20 mg daily	2024-11-03	2024-11-13
	8	8	Omeprazole	40 mg daily	2024-11-08	2024-11-28
	9	9	Citalopram	20 mg daily	2024-11-15	2024-12-15
	10	10	Tamsulosin	0.4 mg daily	2024-11-10	2024-11-30
•	NULL	NULL	NULL	NULL	NULL	NULL

In Bill Table

```
CREATE TABLE Bill (  
    BillID INT AUTO_INCREMENT PRIMARY KEY,  
    PatientID INT NOT NULL,  
    DoctorID INT NOT NULL,  
    BillDate DATE NOT NULL,  
    ConsultationFee DECIMAL(10, 2),  
    MedicationCost DECIMAL(10, 2),  
    OtherCharges DECIMAL(10, 2),  
    TotalAmount DECIMAL(10, 2),  
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),  
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)  
);  
DESC Bill;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

Field	Type	Null	Key	Default	Extra
BillID	int	NO	PRI	NULL	auto_increment
PatientID	int	NO	MUL	NULL	
DoctorID	int	NO	MUL	NULL	
BillDate	date	NO		NULL	
ConsultationFee	decimal(10,2)	YES		NULL	
MedicationCost	decimal(10,2)	YES		NULL	
OtherCharges	decimal(10,2)	YES		NULL	
TotalAmount	decimal(10,2)	YES		NULL	

Performing **DML** Operations



SELECT

The SELECT statement in MySQL is used to query and retrieve data from one or more tables in a database.

Using **Select**, Retrieving the **names** and **phone** numbers of all **male** patients.

```
-- 1.Select --  
-- Retrieving the names and phone numbers of all male patients.  
• SELECT FirstName, LastName, PhoneNumber FROM Patients  
WHERE Gender = 'Male';
```

Result Grid | Filter Rows:

	FirstName	LastName	PhoneNumber
▶	Bob	Williams	2345678901
	Daniel	Brown	4567890123
	Frank	Martinez	6789012345
	Henry	Lee	8901234567
	Jack	Hall	0123456789

INSERT

In MySQL, the INSERT statement is used to add new rows of data to a table.

Using **Insert**,

1. Add a new patient **name** "Liam Green" to the **Patients** table.
2. Assign a **new** appointment for **Patient ID 2** with **Doctor ID 3** on '2024-12-01' for "**Back Pain**".

```
-- 2.Insert --  
-- Operation 1. Add a new patient named "Liam Green" to the Patients table.  
INSERT INTO Patients (FirstName, LastName, Gender, DateOfBirth, PhoneNumber, Address)  
VALUES ('Liam', 'Green', 'Male', '1992-10-15', '1112223333', '123 New St');  
  
-- Operation 2.Assign a new appointment for PatientID 2 with DoctorID 3 on '2024-12-01' for "Back Pain".  
INSERT INTO Appointments (PatientID, DoctorID, AppointmentDate, Reason)  
VALUES (2, 3, '2024-12-01 10:00:00', 'Back Pain');
```

PatientID	FirstName	LastName	Gender	DateOfBirth	PhoneNumber	Address
1	Alice	Johnson	Female	1985-03-15	1234567890	123 Main St
2	Bob	Williams	Male	1990-07-20	2345678901	456 Oak St
3	Catherine	Jones	Female	2000-12-10	3456789012	789 Pine St
4	Daniel	Brown	Male	1975-05-25	4567890123	101 Maple St
5	Eve	Garcia	Female	1988-09-05	5678901234	202 Birch St
6	Frank	Martinez	Male	1965-02-18	6789012345	303 Cedar St
7	Grace	Rodriguez	Female	1995-11-30	7890123456	404 Elm St
8	Henry	Lee	Male	1978-01-22	8901234567	505 Spruce St
9	Isla	Walker	Female	1999-04-14	9012345678	606 Aspen St
10	Jack	Hall	Male	1982-06-09	0123456789	707 Willow St
11	Liam	Green	Male	1992-10-15	1112223333	123 New St

AppointmentID	PatientID	DoctorID	AppointmentDate	Reason
1	1	1	2024-11-21 09:00:00	Routine check-up
2	2	2	2024-11-21 10:00:00	Headache evaluation
3	3	3	2024-11-21 11:00:00	Knee pain
4	4	4	2024-11-21 12:00:00	Child vaccination
5	5	5	2024-11-21 13:00:00	MRI scan
6	6	6	2024-11-21 14:00:00	Cancer consultation
7	7	7	2024-11-21 15:00:00	Skin rash
8	8	8	2024-11-21 16:00:00	Stomach ache
9	9	9	2024-11-21 17:00:00	Depression counseling
10	10	10	2024-11-21 18:00:00	Bladder issues
11	2	3	2024-12-01 10:00:00	Back Pain

UPDATE

The UPDATE statement in MySQL is used to modify existing records in a table.

Using **Update**, Update the **phone number** of a **patient named "Alice Johnson"** to **"9998887777"**.

```
-- 3.Update --  
-- Operation 1.Update the phone number of a patient named "Alice Johnson" to "9998887777".  
UPDATE Patients  
SET PhoneNumber = 9998887777  
WHERE PatientID = 1;
```

Result Grid		Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:	
	PatientID	FirstName	LastName	Gender	DateOfBirth	PhoneNumber	Address		
▶	1	Alice	Johnson	Female	1985-03-15	9998887777	123 Main St		
	2	Bob	Williams	Male	1990-07-20	2345678901	456 Oak St		
	3	Catherine	Jones	Female	2000-12-10	3456789012	789 Pine St		
	4	Daniel	Brown	Male	1975-05-25	4567890123	101 Maple St		
	5	Eve	Garcia	Female	1988-09-05	5678901234	202 Birch St		
	6	Frank	Martinez	Male	1965-02-18	6789012345	303 Cedar St		
	7	Grace	Rodriguez	Female	1995-11-30	7890123456	404 Elm St		
	8	Henry	Lee	Male	1978-01-22	8901234567	505 Spruce St		
	9	Isla	Walker	Female	1999-04-14	9012345678	606 Aspen St		
	10	Jack	Hall	Male	1982-06-09	0123456789	707 Willow St		
	11	Liam	Green	Male	1992-10-15	1112223333	123 New St		
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL		

DELETE

In MySQL, DELETE is a statement used to remove rows from a table based on a specified condition.

Using Delete,

1. Update the **phone number** of a **patient named "Alice Johnson"** to **"9998887777"**.
2. Delete all **appointments scheduled** with Doctor ID 2.

```
-- 4.Delete --
-- Operation 1.Remove a patient record with PatientID 10 from the system.
DELETE FROM Patients
WHERE PatientID = 11;

-- Operation 2.Delete all appointments scheduled with DoctorID 2.
DELETE FROM Appointments
WHERE AppointmentID = 2;
```

Result Grid		Filter Rows:		Edit:	Export/Import:		Wrap Cell Contents:	
	PatientID	FirstName	LastName	Gender	DateOfBirth	PhoneNumber	Address	
▶	1	Alice	Johnson	Female	1985-03-15	9998887777	123 Main St	
	2	Bob	Williams	Male	1990-07-20	2345678901	456 Oak St	
	3	Catherine	Jones	Female	2000-12-10	3456789012	789 Pine St	
	4	Daniel	Brown	Male	1975-05-25	4567890123	101 Maple St	
	5	Eve	Garcia	Female	1988-09-05	5678901234	202 Birch St	
	6	Frank	Martinez	Male	1965-02-18	6789012345	303 Cedar St	
	7	Grace	Rodriguez	Female	1995-11-30	7890123456	404 Elm St	
	8	Henry	Lee	Male	1978-01-22	8901234567	505 Spruce St	
	9	Isla	Walker	Female	1999-04-14	9012345678	606 Aspen St	
	10	Jack	Hall	Male	1982-06-09	0123456789	707 Willow St	
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	

Performing **DDL** Operations



CREATE

In MySQL, the CREATE statement is used to create a new database, table, index, or other objects.

Using **Create**,

1. Create a **table Staff** to store staff details, and add **5 records** to it.

```
-- 1.Create --  
• CREATE TABLE Staff (  
    StaffID INT AUTO_INCREMENT PRIMARY KEY,  
    FirstName VARCHAR(50) NOT NULL,  
    LastName VARCHAR(50) NOT NULL,  
    Role VARCHAR(50) NOT NULL,  
    Gender VARCHAR(10),  
    Salary DECIMAL(10, 2)  
);
```

Result Grid

 Filter Rows:

Edit:   

Export/Import:

	StaffID	FirstName	LastName	Role	Gender	Salary
▶	1	Anna	Johnson	Nurse	Female	50000.00
	2	Mark	Taylor	Technician	Male	45000.00
	3	Sophia	Brown	Receptionist	Female	30000.00
	4	David	Lee	Security	Male	25000.00
	5	Olivia	White	Cleaner	Female	20000.00
*	NULL	NULL	NULL	NULL	NULL	NULL

ALTER

In MySQL, the ALTER statement is used to modify an existing database table.

Using **Alter**,

1. ADD Coleman Staff Education(StaffEID)

```
-- Alter --  
-- Alter the Staff table to add a new column Staff Email Id (StaffEID) --  
• ALTER TABLE Staff  
  ADD StaffEID VARCHAR(50);
```

Result Grid						
Filter Rows:		Edit:		Export/Import:		
StaffID	FirstName	LastName	Role	Gender	Salary	StaffEID
1	Anna	Johnson	Nurse	Female	50000.00	NULL
2	Mark	Taylor	Technician	Male	45000.00	NULL
3	Sophia	Brown	Receptionist	Female	30000.00	NULL
4	David	Lee	Security	Male	25000.00	NULL
5	Olivia	White	Cleaner	Female	20000.00	NULL
*	NULL	NULL	NULL	NULL	NULL	NULL

DORP

In MySQL, **DROP** is a SQL command used to delete a database, table, index, or other objects permanently.

Using **Drop**,

1. Drop the **Staff table** from the database.

```
27      -- Drop --
28 •    DROP TABLE Staff;|
29
```



Output



Action Output



#

Time

Action

1

11:46:21

DROP TABLE Staff



TRUNCATE

In MySQL, **TRUNCATE** is a Data Definition Language (DDL) command used to quickly delete all rows from a table, resetting the auto-increment counter, if any.

Using **Truncate**,

1. **Truncate** the **Staff** table from the database.

```
22 -- Truncate --
23 • TRUNCATE TABLE Staff;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	StaffID	FirstName	LastName	Role	Gender	Salary	StaffID
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Staff 3 x

Output

Action Output

#	Time	Action	Message
✓ 1	13:32:47	TRUNCATE TABLE Staff	0 row(s) affected
✓ 2	13:32:53	SELECT * FROM Staff LIMIT 0, 5000	0 row(s) returned
✓ 3	13:32:53	SELECT * FROM Staff LIMIT 0, 2000	0 row(s) returned
✓ 4	13:32:53	TRUNCATE TABLE Staff	0 row(s) affected

Performing **DQL** Operations



SELECT

In MySQL, the SELECT statement is used to retrieve data from one or more tables.

Using **Select**,
1. **Select** the Appointment.

```
SELECT * FROM Appointments;
```

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	AppointmentID	PatientID	DoctorID	AppointmentDate	Reason
▶	1	1	1	2024-11-21 09:00:00	Routine check-up
	2	2	2	2024-11-21 10:00:00	Headache evaluation
	3	3	3	2024-11-21 11:00:00	Knee pain
	4	4	4	2024-11-21 12:00:00	Child vaccination
	5	5	5	2024-11-21 13:00:00	MRI scan
	6	6	6	2024-11-21 14:00:00	Cancer consultation
	7	7	7	2024-11-21 15:00:00	Skin rash
	8	8	8	2024-11-21 16:00:00	Stomach ache
	9	9	9	2024-11-21 17:00:00	Depression counseling
	10	10	10	2024-11-21 18:00:00	Bladder issues
	11	2	3	2024-12-01 10:00:00	Back Pain
•	NULL	NULL	NULL	NULL	NULL

•

11

5

3

2024-12-01 10:00:00

Back Pain

10

10

10

2024-11-21 18:00:00

Bladder issues

9

9

9

2024-11-21 17:00:00

Depression counseling

Operator



Arithmetic Operators

1. **Addition (+)**: Adds two values together.

-- 1.Add Consultation Fee and Medication Cost

```
SELECT BillID, ConsultationFee + MedicationCost AS TotalCharges
FROM Bill;
```

BillID	TotalCharges
1	150.00
2	225.00
3	300.00
4	180.00
5	200.00
6	270.00
7	205.00
8	165.00
9	245.00
10	265.00

2. **Subtraction (-)**: Subtracts the second value from the first.

-- 2.Subtract Other Charges from Total Amount

```
SELECT BillID, TotalAmount - OtherCharges AS NetAmount
FROM Bill;
```

BillID	NetAmount
1	150.00
2	225.00
3	300.00
4	180.00
5	200.00
6	270.00
7	205.00
8	165.00
9	245.00
10	265.00

3. **Multiplication (*)**: Multiplies two values.

-- 3. Multiply Consultation Fee by a Factor (e.g., 1.1 for a 10% increase)

```
SELECT BillID, ConsultationFee * 1.1 AS UpdatedFee
FROM Bill;
```

BillID	UpdatedFee
1	110.000
2	165.000
3	220.000
4	132.000
5	143.000
6	198.000
7	154.000
8	121.000
9	176.000
10	187.000

4. **Division (/)**: Divides the first value by the second.

-- 4.Divide Total Amount by 2 (e.g., Split Bill Between Two Parties)

```
SELECT BillID, TotalAmount / 2 AS SplitAmount
FROM Bill;
```

BillID	SplitAmount
1	85.000000
2	125.000000
3	175.000000
4	105.000000
5	120.000000
6	165.000000
7	120.000000
8	95.000000
9	145.000000
10	157.500000





	BillID	PatientID	DoctorID	BillDate	ConsultationFee	MedicationCost	OtherCharges	TotalAmount
▶ •	2 NULL	2 NULL	2 NULL	2024-11-19 NULL	150.00 NULL	75.00 NULL	25.00 NULL	250.00 NULL

```
-- 2. Not Equal (!= or <>)  
-- Retrieve bills where the TotalAmount is not equal to 250.00:  
SELECT * FROM Bill WHERE TotalAmount != 250.00;
```



```
SELECT * FROM Bill WHERE MedicationCost < 70.00;
```

4. **>**: Checks if the left value is greater than the right value.

```
SELECT * FROM Bill WHERE OtherCharges > 40.00;
```



```
SELECT * FROM Bill WHERE TotalAmount <= 200.00;
```

6. **>=**: Checks if the left value is greater than or equal to the right value.

```
-- Retrieve bills where the ConsultationFee is greater than or equal to 180.00:
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

Logical Operators

1. **AND**: Returns TRUE if all conditions are true.

-- 1. AND Example: Patients with medication costs over 50 and consultation fees over 100.

```
SELECT * FROM Bill
```

```
WHERE MedicationCost > 50 AND ConsultationFee > 100;
```

Result Grid		Filter Rows:		Edit:		Export/Imports:		Wrap Cell Content:	
BILLID	PatientID	DoctorID	BillDate	ConsultationFee	MedicationCost	OtherCharges	TotalAmount		
2	2	2	2024-11-19	150.00	75.00	25.00	250.00		
3	3	3	2024-11-18	200.00	100.00	50.00	350.00		
4	4	4	2024-11-17	120.00	60.00	30.00	210.00		
5	5	5	2024-11-16	130.00	70.00	40.00	240.00		
6	6	6	2024-11-15	180.00	90.00	60.00	330.00		
7	7	7	2024-11-14	140.00	65.00	35.00	240.00		
8	8	8	2024-11-13	110.00	55.00	25.00	190.00		
9	9	9	2024-11-12	160.00	85.00	45.00	290.00		
10	10	10	2024-11-11	170.00	95.00	50.00	315.00		

2. **OR**: Returns TRUE if any condition is true.

-- 2. OR Example: Patients with either total amount above 300 or other charges above 40.

```
SELECT * FROM Bill
```

```
WHERE TotalAmount > 300 OR OtherCharges > 40;
```

Result Grid		Filter Rows:		Edit:	Export/Import:		Wrap Cell Content:	
	BILLID	PatientID	DoctorID	BillDate	ConsultationFee	MedicationCost	OtherCharges	TotalAmount
▶	3	3	3	2024-11-18	200.00	100.00	50.00	350.00
	6	6	6	2024-11-15	180.00	90.00	60.00	330.00
	9	9	9	2024-11-12	160.00	85.00	45.00	290.00
	10	10	10	2024-11-11	170.00	95.00	50.00	315.00
*	HIDE	HIDE	HIDE	HIDE	HIDE	HIDE	HIDE	HIDE

3. **NOT**: Reverses the result of a condition (TRUE becomes FALSE and vice versa).

-- 3. NOT Example: Bills where other charges are not equal to 50.

```
SELECT * FROM Bill
```

```
WHERE NOT OtherCharges = 50;
```

Result Grid		Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:	
	BILLID	PatientID	DoctorID	BillDate	ConsultationFee	MedicationCost	OtherCharges	TotalAmount	
▶	1	1	1	2024-11-20	100.00	50.00	20.00	170.00	
	2	2	2	2024-11-19	150.00	75.00	25.00	250.00	
	4	4	4	2024-11-17	120.00	60.00	30.00	210.00	
	5	5	5	2024-11-16	130.00	70.00	40.00	240.00	
	6	6	6	2024-11-15	180.00	90.00	60.00	330.00	
	7	7	7	2024-11-14	140.00	65.00	35.00	240.00	
	8	8	8	2024-11-13	110.00	55.00	25.00	190.00	
	9	9	9	2024-11-12	160.00	85.00	45.00	290.00	
▲	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	

Range operator

1. **BETWEEN:** Retrieves values within a given range (inclusive of the boundary values)

```
-- 1. Perform BETWEEN Operation
-- Retrieve bills with total amounts between 200.00 and 300.00.
SELECT BillID, TotalAmount FROM Bill
WHERE TotalAmount BETWEEN 200.00 AND 300.00;
```

Result Grid			Filter Rows:
	BillID	TotalAmount	
▶	2	250.00	
	4	210.00	
	5	240.00	
	7	240.00	
	9	290.00	
✱	NULL	NULL	

2. **NOT BETWEEN:** Retrieves values outside the specified range.

```
-- 2. Perform NOT BETWEEN Operation
-- Retrieve bills with total amounts outside the range 200.00 to 300.00.
SELECT BillID, TotalAmount
FROM Bill
WHERE TotalAmount NOT BETWEEN 200.00 AND 300.00;
```

Result Grid			Filter Rows:
	BillID	TotalAmount	
▶	1	170.00	
	3	350.00	
	6	330.00	
	8	190.00	
	10	315.00	
✱	NULL	NULL	

List operator

1. **IN**: Checks if a value exists in a given list of values.

```
-- 1. IN: Filter Records Based on a List of Values
-- Get bills where the DoctorID is 1, 3, or 5.
SELECT * FROM Bill
WHERE DoctorID IN (1, 3, 5);
```

Result Grid		 Filter Rows:		Edit:   		Export/Import:  		Wrap Cell Content: 	
	BillID	PatientID	DoctorID	BillDate	ConsultationFee	MedicationCost	OtherCharges	TotalAmount	
▶	1	1	1	2024-11-20	100.00	50.00	20.00	170.00	
	3	3	3	2024-11-18	200.00	100.00	50.00	350.00	
	5	5	5	2024-11-16	130.00	70.00	40.00	240.00	
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	

2. **NOT IN**: Checks if a value does not exist in a given list of values.

```
-- 2. NOT IN: Exclude Records Based on a List of Values
-- Get all patients except those with PatientID 2, 4, or 6.
SELECT * FROM Patients
WHERE PatientID NOT IN (2, 4, 6);
```

Result Grid		Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:	
	PatientID	FirstName	LastName	Gender	DateOfBirth	PhoneNumber	Address		
▶	1	Alice	Johnson	Female	1985-03-15	1234567890	123 Main St		
	3	Catherine	Jones	Female	2000-12-10	3456789012	789 Pine St		
	5	Eve	Garcia	Female	1988-09-05	5678901234	202 Birch St		
	7	Grace	Rodriguez	Female	1995-11-30	7890123456	404 Elm St		
	8	Henry	Lee	Male	1978-01-22	8901234567	505 Spruce St		
	9	Isla	Walker	Female	1999-04-14	9012345678	606 Aspen St		
	10	Jack	Hall	Male	1982-06-09	0123456789	707 Willow St		
	11	Liam	Green	Male	1992-10-15	1112223333	123 New St		
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL		



SQL Keywords And Clause



ORDER BY

The ORDER BY clause in SQL is used to sort the result set of a query in either ascending (ASC) or descending (DESC) order based on one or more columns. By default, the sorting order is ascending if not explicitly mentioned.

-- 1. ASC: Ascending order (default).

-- Let's retrieve all records from the Bill table and sort them by TotalAmount in Ascending order.

```
SELECT BillID, PatientID, TotalAmount FROM Bill  
ORDER BY TotalAmount ASC;
```

Result Grid				Filter Rows:	
BillID	PatientID	TotalAmount			
1	1	170.00			
8	8	190.00			
4	4	210.00			
5	5	240.00			
7	7	240.00			
2	2	250.00			
9	9	290.00			
10	10	315.00			
6	6	330.00			
3	3	350.00			
NOLE			NOLE	NOLE	NOLE

-- 2. DESC: Descending order.

-- Let's retrieve all records from the Bill table and sort them by TotalAmount in Descending order.

```
SELECT BillID, PatientID, TotalAmount FROM Bill  
ORDER BY TotalAmount DESC;
```

Result Grid				Filter Rows:	
BillID	PatientID	TotalAmount			
3	3	350.00			
6	6	330.00			
10	10	315.00			
9	9	290.00			
2	2	250.00			
5	5	240.00			
7	7	240.00			
4	4	210.00			
8	8	190.00			
1	1	170.00			
NOLE			NOLE	NOLE	NOLE

DISTINCT Keyword

The DISTINCT keyword in SQL is used to remove duplicate rows from the result set and return only unique values in the specified column(s). It is particularly useful when you want to retrieve a list of unique entries from a column that may contain duplicates.

```
-- From the Bill table, retrieve unique TotalAmount values paid by patients.  
SELECT DISTINCT TotalAmount FROM Bill;
```

TotalAmount
170.00
250.00
350.00
210.00
240.00
330.00
190.00
▶ 290.00
315.00

LIKE operator

The LIKE operator in SQL is used to search for a specified pattern in a column.

1. % Wildcard: Match Zero or More Characters.

-- 1. % Wildcard: Match Zero or More Characters.

-- Find Patients Whose Address Contains 'St':

```
SELECT * FROM Patients WHERE Address LIKE '%St%';
```

PatientID	FirstName	LastName	Gender	DateOfBirth	PhoneNumber	Address
1	Alice	Johnson	Female	1985-03-15	1234567890	123 Main St
2	Bob	Williams	Male	1990-07-20	2345678901	456 Oak St
3	Catherine	Jones	Female	2000-12-10	3456789012	789 Pine St
4	Daniel	Brown	Male	1975-05-25	4567890123	101 Maple St
5	Eve	Garcia	Female	1988-09-05	5678901234	202 Birch St
6	Frank	Martinez	Male	1965-02-18	6789012345	303 Cedar St
7	Grace	Rodriguez	Female	1995-11-30	7890123456	404 Elm St
8	Henry	Lee	Male	1978-01-22	8901234567	505 Spruce St
9	Isla	Walker	Female	1999-04-14	9012345678	606 Aspen St
10	Jack	Hall	Male	1982-06-09	0123456789	707 Willow St
11	Liam	Green	Male	1992-10-15	1112223333	123 New St
*	NULL	NULL	NULL	NULL	NULL	NULL

2. _ Wildcard: Match Exactly One Character

-- 2. _ Wildcard: Match Exactly One Character.

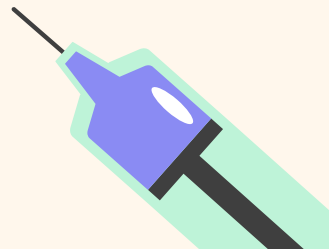
-- Find Patients Whose First Name Ends with 'n':

```
SELECT * FROM Patients WHERE FirstName LIKE '__n%';
```

PatientID	FirstName	LastName	Gender	DateOfBirth	PhoneNumber	Address
6	Frank	Martinez	Male	1965-02-18	6789012345	303 Cedar St
*	NULL	NULL	NULL	NULL	NULL	NULL



SQL Functions



String Function

In MySQL, **string functions** are used to perform operations on string data, such as manipulating or retrieving specific information

1. **CONCAT():** Concatenate two or more strings.

```
76 -- 1.CONCAT() --  
77 • SELECT PatientID, CONCAT(FirstName, ' ', LastName) AS FullName FROM Patients;  
79
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

PatientID	FullName
1	Alice Johnson
2	Bob Williams
3	Catherine Jones
4	Daniel Brown
5	Eve Garcia
6	Frank Martinez
7	Grace Rodriguez
8	Henry Lee
9	Isla Walker
10	Jack Hall

3. **LENGTH():** Get the length of a string.

```
85 -- 4.LENGTH() --  
86 • SELECT PatientID, LENGTH(Address) AS AddressLength FROM Patients;  
97
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

PatientID	AddressLength
1	11
2	10
3	11
4	12
5	12
6	12
7	10
8	13
9	12
10	13

2. **LOWER():** Convert a string to lowercase.

```
82 -- 3.LOWER() --  
83 • SELECT DoctorID, LOWER(LastName) AS LowercaseLastName FROM Doctors;  
94
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

DoctorID	LowercaseLastName
1	doe
2	smith
3	brown
4	davis
5	miller
6	wilson
7	taylor
8	anderson
9	thomas
10	harris

4. **SUBSTRING():** Extract a part of a string.

```
88 -- 5.SUBSTRING() --  
89 • SELECT DoctorID, SUBSTRING(FirstName, 1, 3) AS FirstNameSnippet FROM Doctors;  
90
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

PatientID	AddressLength
1	11
2	10
3	11
4	12
5	12
6	12
7	10
8	13
9	12
10	13

3. **LENGTH()**: Get the length of a string.

```
85      -- 4.LENGTH() --  
86 •    SELECT PatientID, LENGTH(Address) AS AddressLength FROM Patients;  
97
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	PatientID	AddressLength
▶	1	11
	2	10
	3	11
	4	12
	5	12
	6	12
	7	10
	8	13
	9	12
	10	13

4. **SUBSTRING()**: Extract a part of a string.

```
88      -- 5.SUBSTRING() --  
89 •    SELECT DoctorID, SUBSTRING(FirstName, 1, 3) AS FirstNameSnippet FROM Doctors;  
90
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	PatientID	AddressLength
▶	1	11
	2	10
	3	11
	4	12
	5	12
	6	12
	7	10
	8	13
	9	12
	10	13

Math Function

In MySQL, math functions are used to perform mathematical calculations.

1. **ROUND()**: Round to a Specified Number of Decimal Places

```
92 -- 1.Round --
93 • SELECT BillID, ROUND(TotalAmount, 1) AS RoundedAmount
94 FROM Bill;
95
```

BillID	RoundedAmount
1	170.0
2	250.0
3	350.0
4	210.0
5	240.0
6	330.0
7	240.0
8	190.0
9	290.0
10	315.0

2. **CEIL()**: Round Up to the Nearest Integer

```
96 -- 2.Ceil --
97 • SELECT BillID, CEIL(TotalAmount) AS RoundedUpAmount
98 FROM Bill;
99
```

BillID	RoundedUpAmount
1	170
2	250
3	350
4	210
5	240
6	330
7	240
8	190
9	290
10	315

3. **FLOOR()**: Round Down to the Nearest Integer

```
100 -- 3.Floor --
101 • SELECT BillID, FLOOR(TotalAmount) AS RoundedDownAmount
102 FROM Bill;
103
```

BillID	RoundedDownAmount
1	170
2	250
3	350
4	210
5	240
6	330
7	240
8	190
9	290
10	315

4. **POWER()**: Raise a Number to a Power

```
104 -- 4. Power --
105 • SELECT BillID, POWER(ConsultationFee, 2) AS FeeSquare
106 FROM Bill;
107
```

BillID	FeeSquare
1	10000
2	22500
3	40000
4	14400
5	16900
6	32400
7	19600
8	12100
9	25600
10	28900

Aggregate Function

An aggregate function in MySQL performs a calculation on a set of values and returns a single value, commonly used with the GROUP BY clause. Examples include SUM(), AVG(), MAX(), MIN(), and COUNT().

1. Count the Number of Records

```
109 -- COUNT --  
110 • SELECT COUNT(*) AS TotalPatients FROM Patients;  
111
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
TotalPatients			
10			

2. Calculate the Total

```
112 -- SUM --  
113 • SELECT SUM(TotalAmount) AS TotalRevenue FROM Bill;  
114
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
TotalRevenue			
2585.00			

3. Calculate the Average

115 -- AVG --

116 • `SELECT AVG(ConsultationFee) AS AverageConsultationFee FROM Bill;`

117

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

AverageConsultationFee

▶ 146.000000

5. Find the Minimum Value

118 -- MIN --

119 • `SELECT MIN(TotalAmount) AS MinimumTotalAmount FROM Bill;`

120

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

MinimumTotalAmount

▶ 170.00

4. Find the Maximum Value

121 -- MAX --

122 • `SELECT MAX(TotalAmount) AS MaximumTotalAmount FROM Bill;`

123

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

MaximumTotalAmount

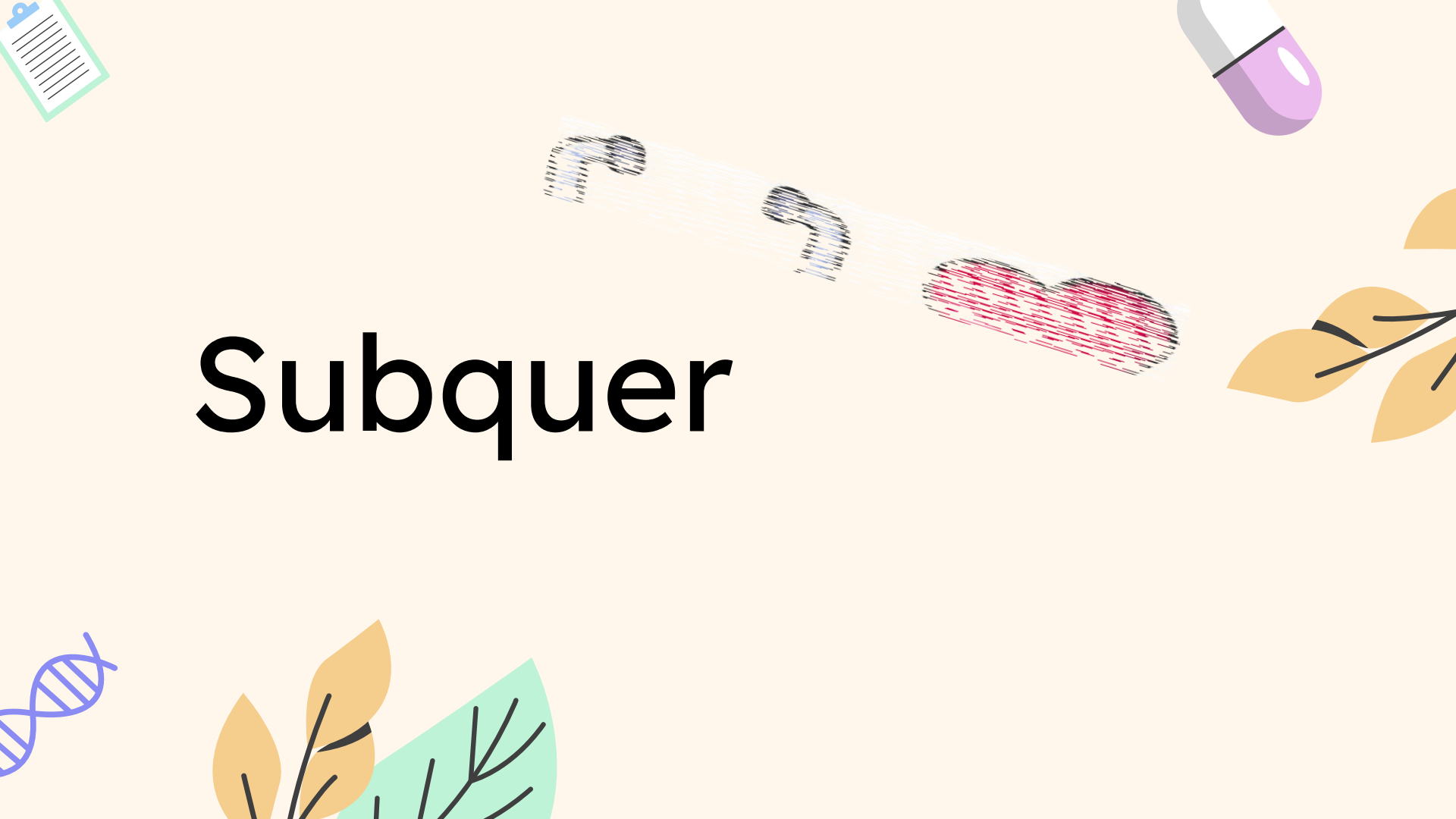
▶ 350.00

GROUP BY with HAVING

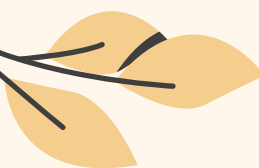
1. Find the total amount spent by each patient and list only those who have spent more than 300.

```
125  -- 1. Total Revenue by Patient
126 • SELECT PatientID, SUM(TotalAmount) AS TotalSpent
127     FROM Bill
128     GROUP BY PatientID
129     HAVING SUM(TotalAmount) > 300;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
PatientID	TotalSpent		
3	350.00		
6	330.00		
10	315.00		



Subquer



A subquery is a nested query within another SQL query. Subqueries can be used in various clauses like SELECT, FROM, WHERE, etc., to perform advanced data retrieval and filtering.

Get each patient's total expenses using a subquery.

```
138 • SELECT PatientID, (SELECT SUM(TotalAmount) FROM Bill
139   WHERE Bill.PatientID = Patients.PatientID) AS TotalExpenses
140   FROM Patients;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
PatientID	TotalExpenses		
1	170.00		
2	250.00		
3	350.00		
4	210.00		
5	240.00		
6	330.00		
7	240.00		
8	190.00		
9	290.00		
10	315.00		



JOINS



SQL joins are used to combine rows from two or more tables based on a related column.

1. **INNER JOIN** : Returns rows where there is a match in both tables.

Retrieve a list of appointments, including the patient's full name, doctor's full name, and the reason for the appointment.

```
143  -- 1.Inner Join
144  •  SELECT Appointments.AppointmentID,
145     CONCAT(Patients.FirstName, ' ', Patients.LastName) AS PatientName,
146     CONCAT(Doctors.FirstName, ' ', Doctors.LastName) AS DoctorName,
147     Appointments.AppointmentDate,
148     Appointments.Reason
149  FROM Appointments
150  INNER JOIN Patients ON Appointments.PatientID = Patients.PatientID
151  INNER JOIN Doctors ON Appointments.DoctorID = Doctors.DoctorID;
152
```

Result Grid					
Filter Rows:		Export:		Wrap Cell Content:	
	AppointmentID	PatientName	DoctorName	AppointmentDate	Reason
▶	1	Alice Johnson	John Doe	2024-11-21 09:00:00	Routine check-up
	2	Bob Williams	Jane Smith	2024-11-21 10:00:00	Headache evaluation
	3	Catherine Jones	Robert Brown	2024-11-21 11:00:00	Knee pain
	4	Daniel Brown	Emily Davis	2024-11-21 12:00:00	Child vaccination
	5	Eve Garcia	Michael Miller	2024-11-21 13:00:00	MRI scan
	6	Frank Martinez	Sarah Wilson	2024-11-21 14:00:00	Cancer consultation
	7	Grace Rodriguez	James Taylor	2024-11-21 15:00:00	Skin rash
	8	Henry Lee	Laura Anderson	2024-11-21 16:00:00	Stomach ache
	9	Isla Walker	David Thomas	2024-11-21 17:00:00	Depression counseling
	10	Jack Hall	Sophia Harris	2024-11-21 18:00:00	Bladder issues

2. LEFT JOIN :

A LEFT JOIN retrieves all rows from the left table and the matching rows from the right table. If there is no match, the result will contain NULL for columns from the right table.

List all doctors along with their patient's details (first name, last name) and the appointment reason. If a doctor has not been assigned to any appointments, still show the doctor's information but with NULL values for the patient's details.

```
154  -- 2.Left Join --
155  •  SELECT
156      Doctors.DoctorID,
157      CONCAT(Doctors.FirstName, ' ', Doctors.LastName) AS DoctorName,
158      CONCAT(Patients.FirstName, ' ', Patients.LastName) AS PatientName,
159      Appointments.Reason
160  FROM Doctors
161  LEFT JOIN Appointments ON Doctors.DoctorID = Appointments.DoctorID
162  LEFT JOIN Patients ON Appointments.PatientID = Patients.PatientID;
163
```

Result Grid	Filter Rows:	Exports	Wrap Cell Contents:
DoctorID	DoctorName	PatientName	Reason
1	John Doe	Alice Johnson	Routine check-up
2	Jane Smith	Bob Williams	Headache evaluation
3	Robert Brown	Catherine Jones	Knee pain
4	Emily Davis	Daniel Brown	Child vaccination
5	Michael Miller	Eve Garcia	MRI scan
6	Sarah Wilson	Frank Martinez	Cancer consultation
7	James Taylor	Grace Rodriguez	Skin rash
8	Laura Anderson	Henry Lee	Stomach ache
9	David Thomas	Isla Walker	Depression counseling
10	Sophia Harris	Jack Hall	Bladder issues

3. RIGTH JOIN :

Returns all rows from the right table and matching rows from the left table. Non-matching rows from the left table will have NULL.

We want to list all medications prescribed to patients and the patient details associated with each medication. If a medication is not linked to a patient (for some reason), we still want to include the medication details, but with NULL values for patient details.

```
164  -- Right JOIN --
165  •  SELECT
166      Medications.MedicationID,
167      Medications.MedicationName,
168      Medications.Dosage,
169      Medications.StartDate,
170      Medications.EndDate,
171      CONCAT(Patients.FirstName, ' ', Patients.LastName) AS PatientName
172  FROM Medications
173  RIGHT JOIN Patients ON Medications.PatientID = Patients.PatientID;
```

Result Grid						
Filter Rows:						
Export:						
Wrap Cell Content:						
	MedicationID	MedicationName	Dosage	StartDate	EndDate	PatientName
▶	1	Aspirin	100 mg daily	2024-11-01	2024-11-30	Alice Johnson
	2	Ibuprofen	200 mg twice daily	2024-11-05	2024-11-20	Bob Williams
	3	Paracetamol	500 mg every 6 hours	2024-11-10	2024-11-15	Catherine Jones
	4	Amoxicillin	250 mg three times daily	2024-11-12	2024-11-22	Daniel Brown
	5	Metformin	500 mg daily	2024-11-18	2024-12-18	Eve Garcia
	6	Atorvastatin	10 mg daily	2024-11-01	2024-12-01	Frank Martinez
	7	Prednisone	20 mg daily	2024-11-03	2024-11-13	Grace Rodriguez
	8	Omeprazole	40 mg daily	2024-11-08	2024-11-28	Henry Lee
	9	Citalopram	20 mg daily	2024-11-15	2024-12-15	Isla Walker
	10	Tamsulosin	0.4 mg daily	2024-11-10	2024-11-30	Jack Hall

4.FULL JOIN:

Combines results of both LEFT JOIN and RIGHT JOIN. Returns all rows when there is a match in one of the tables.

We want to create a list of all patients and doctors. We'll use a UNION to combine their names (from both tables) into a single result. We'll select the first name and last name from both the Patients table and the Doctors table, and merge the two results into a single list.

```
175 -- Full Join --
176 • SELECT CONCAT(FirstName, ' ', LastName) AS Name, 'Patient' AS Role
177 FROM Patients
178 UNION
179 SELECT CONCAT(FirstName, ' ', LastName) AS Name, 'Doctor' AS Role
180 FROM Doctors;
181
182
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	Name	Role		
▶	Alice Johnson	Patient		
	Bob Williams	Patient		
	Catherine Jones	Patient		
	Daniel Brown	Patient		
	Eve Garcia	Patient		
	Frank Martinez	Patient		
	Grace Rodriguez	Patient		
	Henry Lee	Patient		
	Isla Walker	Patient		
	Jack Hall	Patient		
	John Doe	Doctor		
	Jane Smith	Doctor		
	Robert Brown	Doctor		
	Emily Davis	Doctor		
	Michael Miller	Doctor		
	Sarah Wilson	Doctor		



VIWE

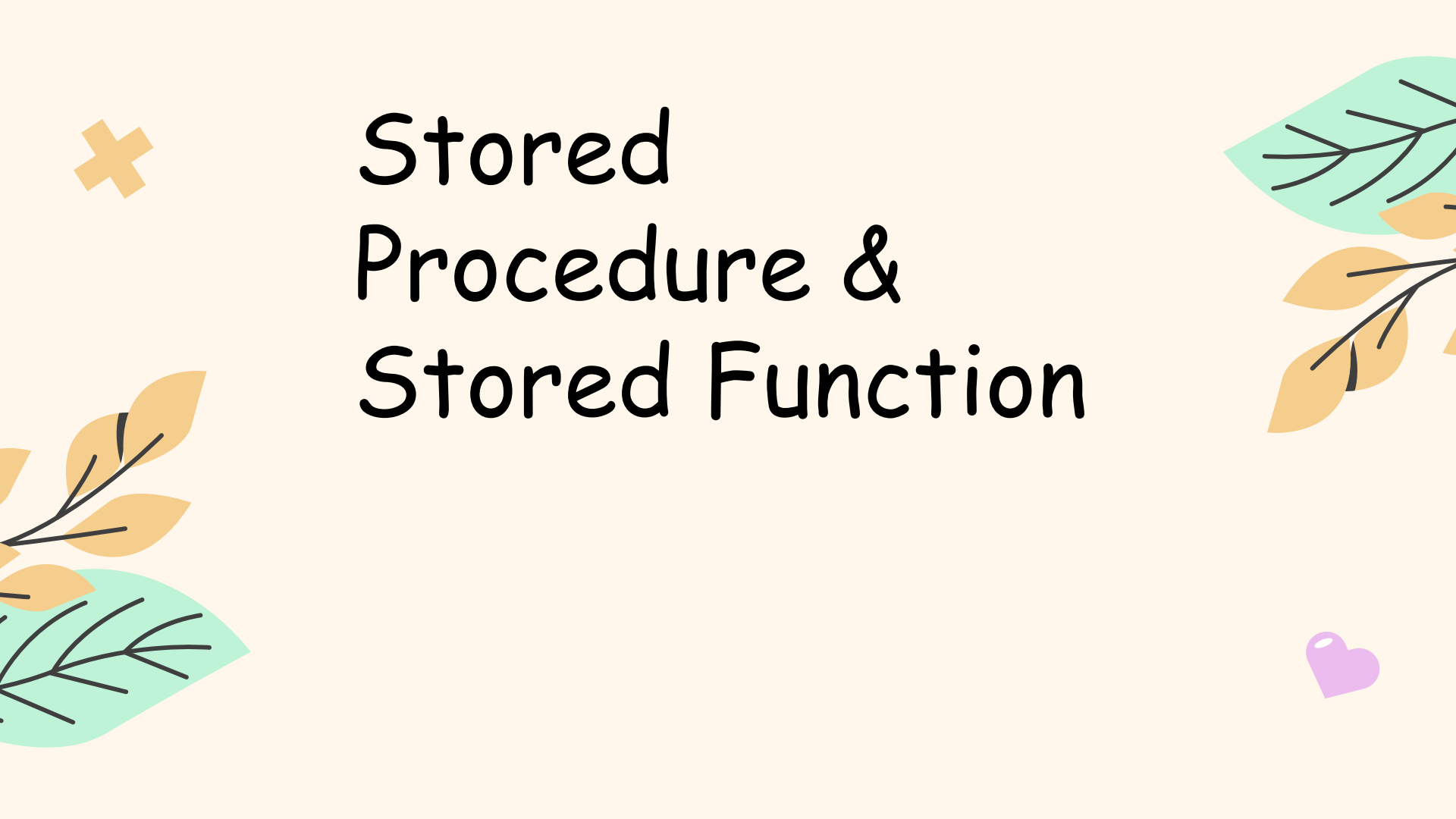
VIEW

A **view** in SQL is a virtual table that provides a way to store a query for future use. It is not a physical table but rather a stored SQL query that can be treated like a table in SELECT statements.

Scenario : Let's create a view that combines information from the **Patients** and **Doctors** tables to show a list of appointments, including the patient's name, doctor's name, and the appointment date.

```
182  -- View --
183  • CREATE VIEW AppointmentDetails AS
184  SELECT
185      CONCAT(Patients.FirstName, ' ', Patients.LastName) AS PatientName,
186      CONCAT(Doctors.FirstName, ' ', Doctors.LastName) AS DoctorName,
187      Appointments.AppointmentDate,
188      Appointments.Reason
189  FROM Appointments
190  JOIN Patients ON Appointments.PatientID = Patients.PatientID
191  JOIN Doctors ON Appointments.DoctorID = Doctors.DoctorID;
192  • SELECT * FROM AppointmentDetails;
```

Result Grid				
Filter Rows: [X]				
Export: [X] Wrap Cell Content: [X]				
	PatientName	DoctorName	AppointmentDate	Reason
▶	Alice Johnson	John Doe	2024-11-21 09:00:00	Routine check-up
	Bob Williams	Jane Smith	2024-11-21 10:00:00	Headache evaluation
	Catherine Jones	Robert Brown	2024-11-21 11:00:00	Knee pain
	Daniel Brown	Emily Davis	2024-11-21 12:00:00	Child vaccination
	Eve Garcia	Michael Miller	2024-11-21 13:00:00	MRI scan
	Frank Martinez	Sarah Wilson	2024-11-21 14:00:00	Cancer consultation
	Grace Rodriguez	James Taylor	2024-11-21 15:00:00	Skin rash
	Henry Lee	Laura Anderson	2024-11-21 16:00:00	Stomach ache
	Isla Walker	David Thomas	2024-11-21 17:00:00	Depression counseling
	Jack Hall	Sophia Harris	2024-11-21 18:00:00	Bladder issues



Stored Procedure & Stored Function

Stored

A **Stored Procedure** is a precompiled collection of SQL statements and optional control-flow logic stored in a relational database.

Insert a new patient and return the PatientID

```
-- Stored Procedure: Insert a new patient and return the PatientID
DELIMITER //

CREATE PROCEDURE AddPatient (
    IN p_FirstName VARCHAR(50),
    IN p_LastName VARCHAR(50),
    IN p_Gender VARCHAR(10),
    IN p_DateOfBirth DATE,
    IN p_PhoneNumber VARCHAR(15),
    IN p_Address TEXT,
    OUT p_PatientID INT
)
BEGIN
    INSERT INTO Patients (FirstName, LastName, Gender, DateOfBirth, PhoneNumber, Address)
    VALUES (p_FirstName, p_LastName, p_Gender, p_DateOfBirth, p_PhoneNumber, p_Address);
    SET p_PatientID = LAST_INSERT_ID();
END //
DELIMITER ;

CALL AddPatient(
    'Alisa',           -- FirstName
    'Wonder',         -- LastName
    'Female',          -- Gender
    '1999-05-17',      -- DateOfBirth
    '9087654890',      -- PhoneNumber
    '156 Kly WSt',     -- Address
    @PatientID         -- Output Parameter to hold PatientID
);
```

Result Grid						
Filter Rows:						
Edit:   						
Export/Imports:  						
Wrap Cell Contents						
PatientID	FirstName	LastName	Gender	DateOfBirth	PhoneNumber	Address
1	Alice	Johnson	Female	1985-03-15	1234567890	123 Main St
2	Bob	Williams	Male	1990-07-20	2345678901	456 Oak St
3	Catherine	Jones	Female	2000-12-10	3456789012	789 Pine St
4	Daniel	Brown	Male	1975-05-25	4567890123	101 Maple St
5	Eve	Garcia	Female	1988-09-05	5678901234	202 Birch St
6	Frank	Martinez	Male	1965-02-18	6789012345	303 Cedar St
7	Grace	Rodriguez	Female	1995-11-30	7890123456	404 Elm St
8	Henry	Lee	Male	1978-01-22	8901234567	505 Spruce St
9	Isla	Walker	Female	1999-04-14	9012345678	606 Aspen St
10	Jack	Hall	Male	1982-06-09	0123456789	707 Willow St
13	Alisa	Wonder	Female	1999-05-17	9087654890	156 Kly WSt
*	NULL	NULL	NULL	NULL	NULL	NULL

Stored Function

A **Stored Function** in MySQL is a reusable SQL code block that performs a specific task and returns a single value.

Calculate a patient's age

```
43  -- Stored Function: Calculate a patient's age
44  DELIMITER //
45  • CREATE FUNCTION CalculateAge(p_DateOfBirth DATE)
46    RETURNS INT
47    DETERMINISTIC
48  BEGIN
49      DECLARE age INT;
50      SET age = TIMESTAMPDIFF(YEAR, p_DateOfBirth, CURDATE());
51      RETURN age;
52  END //
```

Result Grid



Filter Rows:

Export:



Wrap Cell Content:



Age
34



QL Cursor



In MySQL, a **cursor** is a database object that allows row-by-row processing of a result set.

The scenario will involve retrieving total consultation fees for a specific doctor over a date range.

```
DELIMITER //
• CREATE PROCEDURE GetDoctorFees (
  IN doctorID INT,
  IN startDate DATE,
  IN endDate DATE,
  OUT totalFees DECIMAL(10, 2)
)
• BEGIN
  DECLARE fee DECIMAL(10, 2);
  DECLARE done INT DEFAULT FALSE;
  DECLARE cur CURSOR FOR
    SELECT ConsultationFee FROM Bill
    WHERE DoctorID = doctorID AND BillDate BETWEEN startDate AND endDate;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
  SET totalFees = 0;
  OPEN cur;
  read_loop: LOOP
    FETCH cur INTO fee;
    IF done THEN
      LEAVE read_loop;
    END IF;
    SET totalFees = totalFees + fee;
  END LOOP;
  CLOSE cur;
• END //
DELIMITER ;
```

```
85 • CALL GetDoctorFees(1, '2024-11-01', '2024-11-30', @totalFees);
```

```
86 • SELECT @totalFees AS TotalConsultationFees;
```

```
87
```

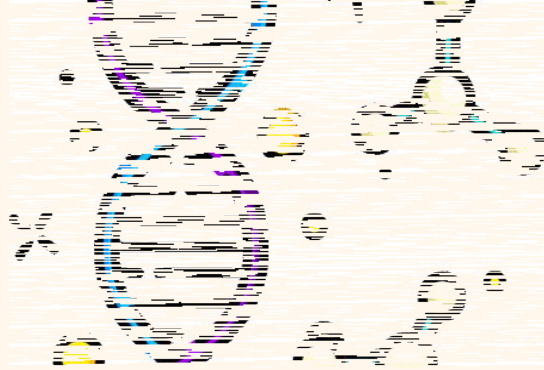
Result Grid

Filter Rows:

Exports

Wrap Cell Contents

TotalConsultationFees
1460.00



SQL Trigger



A trigger in MySQL is a database object that is automatically executed or fired when a specified event occurs on a table.

AFTER :An AFTER trigger executes after the specified event has occurred.

```
CREATE TABLE PatientLogsUpdate (
  LogID INT AUTO_INCREMENT PRIMARY KEY,
  PatientID INT NOT NULL,
  ChangeTime DATETIME NOT NULL,
  OldData TEXT,
  NewData TEXT
);

DELIMITER //

CREATE TRIGGER PatientUpdateLog
AFTER UPDATE ON Patients
FOR EACH ROW
BEGIN
  INSERT INTO PatientLogsUpdate (PatientID, ChangeTime, OldData, NewData)
  VALUES (OLD.PatientID, NOW(), CONCAT(OLD.FirstName, ' ', OLD.LastName), CONCAT(NEW.FirstName, ' ', NEW.LastName));
END //

DELIMITER ;

UPDATE Patients
SET FirstName = 'Alisa', LastName = 'Tina'
WHERE PatientID = 13;

SELECT * FROM PatientLogsUpdate;
```

LogID	PatientID	ChangeTime	OldData	NewData
1	13	2025-01-04 11:47:31	Alisa Wonder	Alisa Tina

BEFORE: A BEFORE trigger executes before the specified event occurs.

```
CREATE TABLE IF NOT EXISTS PatientLogs (
  LogID INT AUTO_INCREMENT PRIMARY KEY,
  PatientID INT,
  LogTime DATETIME,
  Data TEXT
);

-- 2. Create BEFORE INSERT trigger for Patients table
DELIMITER //
CREATE TRIGGER BeforePatientInsertLog
BEFORE INSERT ON Patients
FOR EACH ROW
BEGIN
  INSERT INTO PatientLogs (PatientID, LogTime, Data)
  VALUES (
    NEW.PatientID,
    NOW(),
    CONCAT('Attempting to insert: ',
    NEW.FirstName, ' ', NEW.LastName, ', Gender: ', NEW.Gender, ', DOB: ', NEW.DateOfBirth, ', Phone: ',
    NEW.PhoneNumber, ', Address: ',
    NEW.Address)
  );
END //
DELIMITER ;

INSERT INTO Patients (FirstName, LastName, Gender, DateOfBirth, PhoneNumber, Address)
VALUES ('Braow', 'boster', 'male', '1995-05-10', '2463793562', '445 Hbas str');

SELECT * FROM PatientLogs;
```

LogID	PatientID	LogTime	Data
1	0	2025-01-04 11:55:38	Attempting to insert: Braow boster, G...



**Thank
You.....**

